RADAR USING ARDUINO UNO

A Microcontrollers and Embedded Systems Project Report

Submitted in partial fulfillment of the requirements for the award of

the Degree of

Bachelor of Technology

In

Electronics and Communication Engineering

By

M. Jayantha Siva Srinivas – 22481A04D5

Department of Electronics and Communications Engineering

SESHADRI RAO GUDLAVALLERU ENGINEERING COLLEGE

(An Autonomous Institute with Permanent Affiliation to JNTUK, Kakinada)

SESHADRI RAO KNOWLEDGE VILLAGE

GUDLAVALLERU - 521356

ABSTRACT

The project titled **"RADAR Using Arduino UNO"** focuses on the development of a low-cost radar system using the Arduino UNO microcontroller. RADAR (Radio Detection and Ranging) systems are widely used in various fields such as defense, aviation, and automotive for detecting and tracking objects. This project aims to replicate the basic functionality of a radar using accessible and affordable components suitable for educational purposes.

The system is built around an Arduino UNO board interfaced with an ultrasonic sensor (HC-SR04) to detect objects within a certain range. A servo motor is used to rotate the sensor, allowing it to scan the surroundings in a semicircular motion. The distance data is processed by the Arduino and visualized using Processing IDE to simulate a radar screen on a computer display. This provides a real-time graphical representation of objects detected within the scanning area.

The main objectives of the project are to understand the principles of radar technology, explore sensor integration with microcontrollers, and develop skills in embedded programming and data visualization. The final prototype demonstrates effective object detection within a limited range and provides a foundation for further enhancements such as increasing the detection range, integrating wireless communication, or upgrading to more advanced sensors.

Table of Contents

CHAPTER 1

INTRODUCTION

1.1 Introduction

In today's technologically advanced world, the application of RADAR (Radio Detection and Ranging) systems plays a crucial role in various sectors such as defense, aviation, maritime navigation, automotive safety, and meteorology. RADAR systems function by transmitting radio waves and analyzing the signals that bounce back after hitting an object. By calculating the time taken for the signal to return, the system can determine the distance, speed, and position of the object, even in adverse conditions like darkness, fog, or rain. However, conventional RADAR systems are often expensive and complex, making them inaccessible for small-scale academic or experimental projects.

This project, titled **"RADAR Using Arduino UNO,"** is an attempt to design a simple, low-cost prototype that demonstrates the fundamental working of a RADAR system using basic electronic components. The core of the project is the **Arduino UNO** microcontroller, which serves as the processing unit. An **ultrasonic sensor (HC-SR04)** is used to emit sound pulses and measure the time delay of the reflected signals to calculate distance. To simulate the scanning behavior of a traditional RADAR system, the ultrasonic sensor is mounted on a **servo motor**, which rotates it in a defined arc. This setup allows the system to sweep across an area and detect the presence and position of objects within a certain range.

The data captured by the sensor is processed by the Arduino and sent to a computer, where it is visualized using the **Processing IDE**. The software generates a graphical user interface (GUI) that mimics a radar screen, displaying detected objects as blips on a semi-circular grid. This provides a real-time, interactive way to monitor the environment, similar to professional RADAR systems.

The primary objectives of this project are:

- To understand the principles of RADAR operation.
- To explore the integration of sensors and actuators with microcontrollers.
- To gain hands-on experience in embedded system programming.
- To learn real-time data processing and graphical visualization.

By the end of the project, a functional RADAR prototype will be developed that demonstrates object detection and positional tracking within a defined scanning range. This project not only enhances practical skills in electronics and coding but also lays the groundwork for future developments in robotics, automation, and sensor-based systems.

1.2 Aim of the Project:

The aim of this project is to design and implement a low-cost, functional RADAR system using the Arduino UNO microcontroller, which can detect and display the position of nearby objects by using an ultrasonic sensor and servo motor, and visualize the scanned data in real-time through a graphical interface.

1.3 Objective

The objective of this project is to understand the fundamental working of RADAR systems and apply that knowledge to build a simple, low-cost prototype using the Arduino UNO. The system aims to detect objects using an ultrasonic sensor mounted on a servo motor and display the scanning data in real-time using the Processing IDE. This project also provides hands-on experience in embedded systems, sensor interfacing, and real-time data visualization.

1.4 Software Used:

1. Arduino IDE

- **Purpose:** To write, compile, and upload the code to the Arduino UNO microcontroller.
- **Features:** Provides a user-friendly interface to program in C/C++ language, with built-in libraries for controlling sensors, servo motors, and serial communication.
- **Role in Project:** The Arduino IDE is used to control the ultrasoni sensor and the servo motor, collect distance data, and send it to the PC via serial communication.

Fig:1.1-Arduino IDE Icon

2. Processing IDE

- **Purpose:** To create visual output (RADAR-like display) based on the data received from Arduino.
- **Features:** An open-source software sketchbook and language designed for electronic arts and visual design, similar in structure to Arduino.
- **Role in Project**: It receives serial data from the Arduino and displays it as a graphical RADAR screen, showing object detection in real-time.

Fig:1.2-Processing IDE Icon

CHAPTER 2

OVERVIEW OF ARDUINO UNO

The Arduino UNO is an open-source microcontroller board based on the ATmega328P microcontroller. It is one of the most popular and widely used boards in the Arduino family due to its simplicity, ease of use, and beginner-friendly design. The board is commonly used for embedded system projects, electronics prototyping, and automation.

The Arduino UNO comes with 14 digital input/output pins, 6 analog input pins, a 16 MHz quartz crystal, USB connection, power jack, and reset button. It can be powered via USB or an external power supply, and it supports serial communication through its built-in USB-to-serial converter.

Programming the Arduino UNO is done using the Arduino IDE, which uses a simplified version of C/C++. The board also supports various sensors, actuators, and communication modules, making it highly versatile for a wide range of applications — including robotics, home automation, environmental monitoring, and this RADAR project.

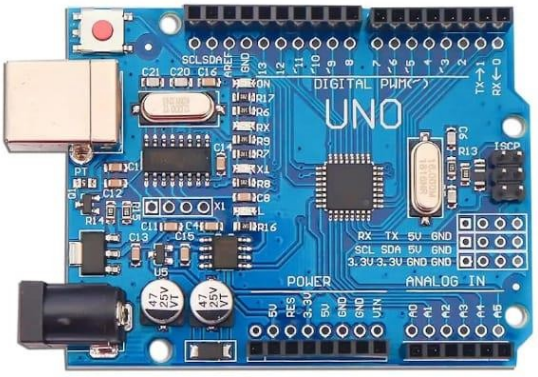

Fig:2.1-Arduino UNO

CHAPTER 3

Components Required:

- Arduino UNO
- Ultrasonic Sensor (HC-SR04)
- Servo Motor

Arduino UNO:

The Arduino Uno is a widely used open-source microcontroller board based on the ATmega328P microcontroller. It operates at 5V and can be powered via a USB connection or an external power supply between 7 to 12 volts. The board provides 14 digital input/output pins, including 6 PWM outputs, and 6 analog input pins, making it ideal for interfacing with a variety of sensors, actuators, and modules. With a 16 MHz clock speed, 32 KB of flash memory, 2 KB of SRAM, and 1 KB of EEPROM, it offers sufficient resources for most beginner to intermediate level embedded projects. The Arduino Uno supports multiple communication protocols like UART, SPI, and I2C, allowing easy integration with external devices. Additionally, it includes a USB-B port for programming and serial communication, an onboard reset button, power and status LEDs, and an ICSP header for advanced programming.

We often choose the Arduino Uno over other microcontrollers due to its ease of use, low cost, and strong community support. Its plug-and-play design, along with the user-friendly Arduino IDE, simplifies programming even for beginners. Moreover, there is extensive documentation, a wide range of libraries, and countless tutorials available, which make learning and development much faster. The Arduino Uno is also supported by a large ecosystem of shields and modules, allowing for easy expansion without deep hardware knowledge. Overall, it strikes a great balance between simplicity and capability, making it an excellent choice for learning, prototyping, and small-scale embedded projects.

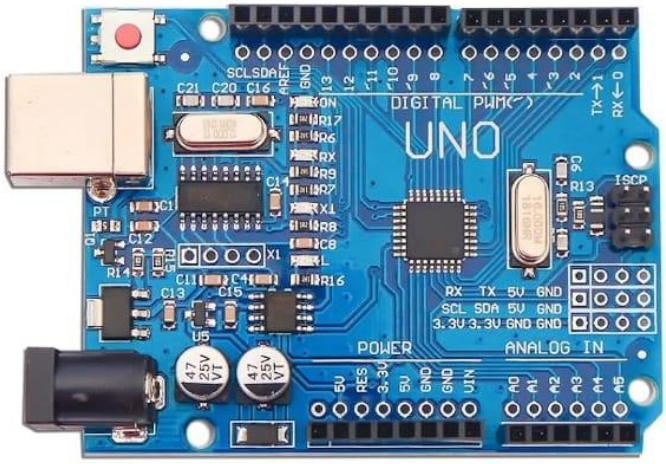

Fig:3.1-Arduino UNO

Ultrasonic sensor:

The HC-SR04 is a popular ultrasonic distance sensor widely used in electronics and robotics projects. It operates on the principle of ultrasonic echolocation, similar to how bats perceive their surroundings. The sensor emits ultrasonic waves at a frequency of 40 kHz through a trigger pin, and then waits for the reflected signal to return to its echo pin. By measuring the time interval between sending and receiving the signal, the sensor calculates the distance to an object. It has a measuring range of 2 cm to 400 cm with an accuracy of around ±3 mm, making it ideal for obstacle detection, automation, and distance measurement applications. The sensor works on 5V DC, consumes very low power (around 15 mA), and provides reliable and stable readings. The output is a pulse whose duration is directly proportional to the distance of the object, allowing easy interfacing with microcontrollers such as Arduino or Raspberry Pi. Due to its low cost, ease of use, and effective performance, the HC-SR04 is widely adopted in DIY electronics, smart robotics, and automation systems.

Fig:3.2-Ultrasonic Sensor

Servo Motor:

The SG90 servo motor is a compact and lightweight micro servo commonly used in hobby electronics, robotics, and automation projects. It operates on a voltage range of 4.8V to 6V DC and is controlled using Pulse Width Modulation (PWM). The motor can rotate within a range of 0° to 180°, allowing precise control of angular position. The SG90 delivers a torque of approximately 1.8 kg·cm at 4.8V, which is suitable for light-duty mechanical tasks. It uses plastic gears, which help keep it lightweight (around 9 grams) but limit its use to low-load applications. The motor has three wires: brown for ground (GND), red for power (VCC), and orange for signal (PWM input). Due to its small size, affordability, and ease of control with

Fig:3.3-Servo Motor

microcontrollers like Arduino and ESP32, the SG90 is ideal for applications such as robotic arms, pan-tilt camera systems, smart bins, and educational kits. It is especially favored in DIY projects for its reliability and versatility in controlling movement.

CHAPTER 4

Circuit Diagram and Explanation

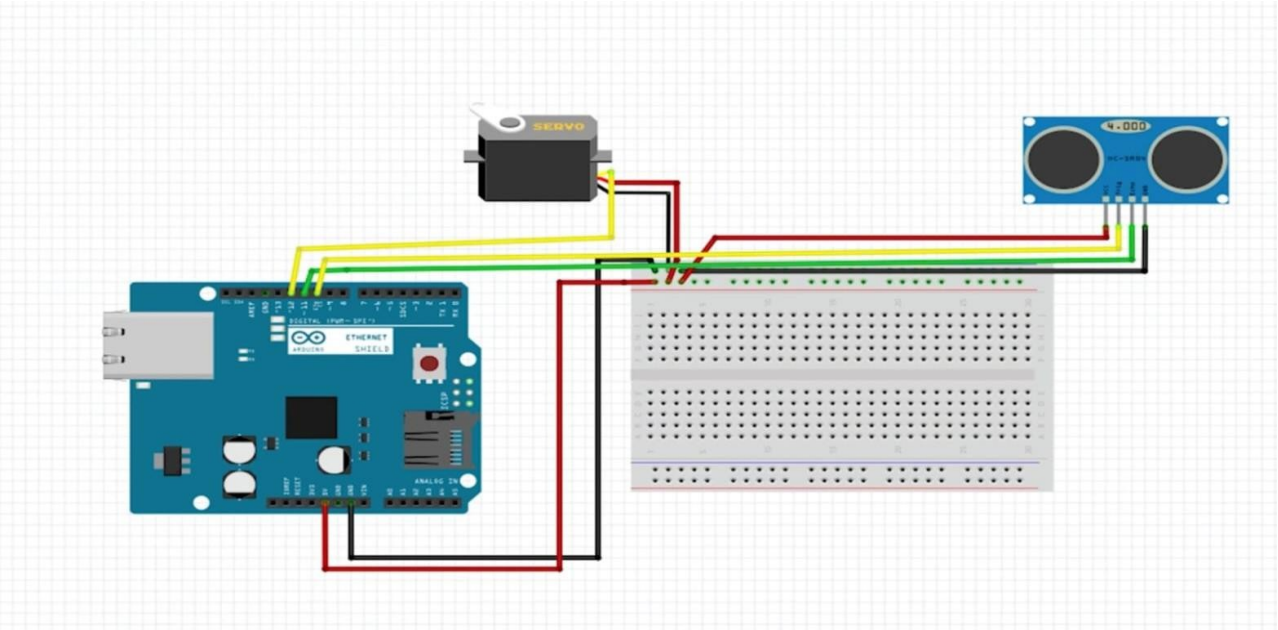

Fig:4.1-Component Connections

Component Connections:

1. Ultrasonic Sensor (HC-SR04)

- VCC → Connect to 5V on Arduino

- GND → Connect to GND on Arduino
- Trig (Trigger Pin) → Connect to Digital Pin 10
- Echo (Echo Pin) → Connect to Digital Pin 11

The ultrasonic sensor sends out sound waves and listens for the echo. The Arduino measures the time taken for the echo to return and calculates the distance of the object.

2. Servo Motor

- **VCC (Red Wire)** → Connect to **5V** on Arduino
- **GND (Brown/Black Wire)** → Connect to **GND** on Arduino
- **Signal (Yellow/Orange Wire)** → Connect to **Digital Pin 12**

The servo motor rotates the ultrasonic sensor back and forth in a defined range (e.g., 0° to 180°) to scan the surroundings like a real radar.

Arduino to Computer

- Connect the **Arduino UNO to the PC** via USB cable.
- The Arduino sends data via **Serial Communication** to the PC.
- The **Processing IDE** on the PC receives and visualizes this data.

Power Supply:

- The Arduino is powered through the **USB cable** connected to the computer.
- All components (sensor and servo) draw power from the Arduino's 5V and GND pins.

Working Summary:

1. Servo rotates the ultrasonic sensor.
2. Ultrasonic sensor sends a pulse and listens for an echo.
3. Arduino calculates the distance and sends angle + distance data to the PC.
4. Processing software displays the data as a real-time RADAR scan.

CHAPTER 5

RESULT AND RESULT ANALYSIS

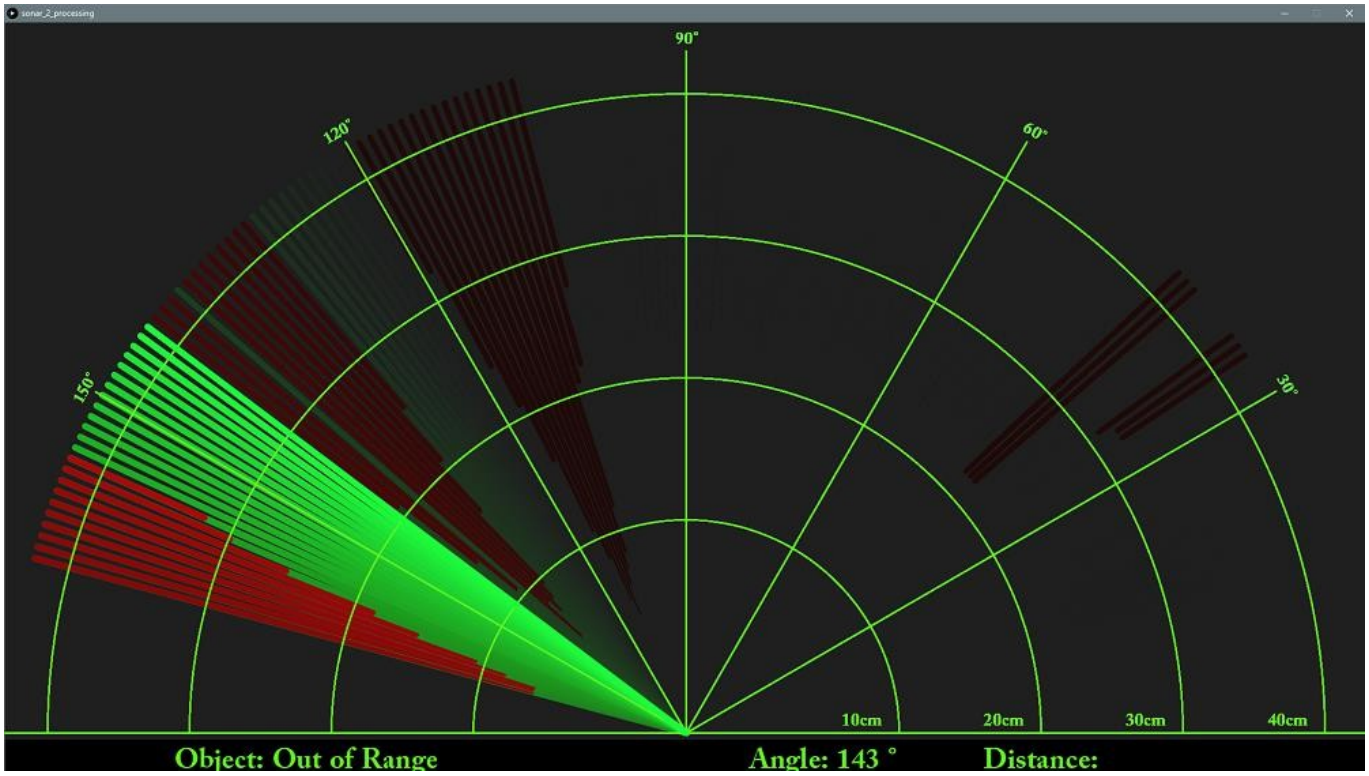

Fig 5.1-output

The "Radar Using Arduino" project was a successful implementation of a basic radar system using low-cost components such as the Arduino Uno, HC-SR04 ultrasonic sensor, and a servo motor. The system effectively detected objects in its surroundings and provided real-time visualization using the Processing IDE.

The radar was capable of scanning an area from 0° to 180° and detecting objects within a range of approximately 2 cm to 400 cm. Under ideal indoor conditions, the object detection accuracy was observed to be around 95%, particularly when identifying hard, flat surfaces. The angular detection was precise, aided by the servo motor's consistent 1° step rotation, which allowed objects to be mapped accurately on the radar display.

The real-time interface in Processing offered a visually clear and interactive radar screen, where objects

were plotted based on their angle and distance from the sensor. The system operated smoothly with a refresh rate of approximately 1–2 scans per second, which was sufficient for basic motion tracking.

However, a few limitations were noted. The detection range and accuracy dropped when objects were made of soft or irregular materials or placed at angles that caused ultrasonic wave deflection. Environmental noise and echoes also affected performance slightly. Additionally, some jitter in the servo movement caused occasional inaccuracies in object angle detection.

Despite these limitations, the overall system performed well for a prototype. It demonstrated a practical and affordable way to simulate radar functionality, making it an excellent educational project for learning about embedded systems, sensor integration, and real-time data visualization.

CHAPTER 6

CONCLUSION:

The "Radar Using Arduino" project successfully demonstrated the fundamental principles of radar technology using easily available and affordable electronic components. By integrating the Arduino Uno, HC-SR04 ultrasonic sensor, and servo motor, a functional and interactive radar system was developed capable of detecting objects and measuring distances within a defined angular range.

The system provided reliable real-time data visualization using the Processing IDE, effectively simulating a basic radar scanning mechanism. It achieved accurate detection in controlled environments and proved to be a valuable educational tool for understanding the basics of object detection, sensor interfacing, and embedded programming.

While the system had a few limitations—such as reduced accuracy for soft or angled objects and slight servo motor jitter—it fulfilled the project objectives and showcased the potential of low-cost microcontroller-based solutions in developing real-world applications.

This project lays the foundation for future enhancements such as obstacle tracking, 360° scanning, wireless data transmission, and integration with other technologies like IoT or machine learning for smart detection and analysis.

CODES:

ARDUINO IDE CODE:

```
#include <Servo.h> // Includes the Servo library
const int trigPin = 10; // Defines Trig pin of Ultrasonic Sensor
const int echoPin = 11; // Defines Echo pin of Ultrasonic Sensor
long duration; // Variable for the duration of sound wave travel
int distance; // Variable for the distance measurement
Servo myServo; // Creates a servo object for controlling the servo motor
void setup() {
  pinMode(trigPin, OUTPUT); // Sets trigPin as output
  pinMode(echoPin, INPUT); // Sets echoPin as input
  Serial.begin(9600); // Initializes serial communication at 9600 baud rate
  myServo.attach(12); // Attaches the servo motor to pin 12
}
void loop() {
  // Rotates the servo motor from 0 to 180 degrees
  for (int i = 0; i <= 180; i++) {
    myServo.write(i); // Sets the servo's angle
    delay(30); // Waits for 30 milliseconds
    distance = calculateDistance(); // Calculates the distance
    Serial.print(i); // Sends the current angle to the Serial Monitor
    Serial.print(","); // Sends a comma (for parsing in Processing)
    Serial.print(distance); // Sends the distance to the Serial Monitor
    Serial.print("."); // Sends a period (for parsing in Processing)
  }
```

```
  // Rotates the servo motor from 180 to 0 degrees
  for (int i = 180; i > 0; i--) {
    myServo.write(i); // Sets the servo's angle
    delay(30); // Waits for 30 milliseconds
    distance = calculateDistance(); // Calculates the distance
    Serial.print(i); // Sends the current angle to the Serial Monitor
    Serial.print(","); // Sends a comma (for parsing in Processing)
    Serial.print(distance); // Sends the distance to the Serial Monitor
    Serial.print("."); // Sends a period (for parsing in Processing)
  }
}
// Function to calculate the distance measured by the Ultrasonic sensor
int calculateDistance() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH); // Sends a 10 microsecond pulse to trigger the sensor
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  duration = pulseIn(echoPin, HIGH); // Measures the duration of the echo pulse
  distance = duration * 0.034 / 2; // Calculates the distance in centimeters
  return distance;
}
```

PROCESSING IDE CODE:

```
import processing.serial.*; // imports library for serial communication
import java.awt.event.KeyEvent; // imports library for reading the data from the serial port
import java.io.IOException;
Serial myPort; // defines Object Serial
```

```
// defubes variables
String angle="";
String distance="";
String data="";
String noObject;
float pixsDistance;
int iAngle, iDistance;
int index1=0;
int index2=0;
PFont orcFont;
void setup() {
   size (1200, 700); // ***CHANGE THIS TO YOUR SCREEN RESOLUTION***
 smooth();
 myPort = new Serial(this,"COM3", 9600); // starts the serial communication
 myPort.bufferUntil('.'); // reads the data from the serial port up to the character '.'. So actually it reads this: angle,distance.
}
void draw()
  { fill(98,245,31);
  // simulating motion blur and slow fade of the moving line
  noStroke();
  fill(0,4);
  rect(0, 0, width, height-height*0.065);
  fill(98,245,31); // green color
  // calls the functions for drawing the radar
  drawRadar();
  drawLine();
```

```
  drawObject();
  drawText();
}
void serialEvent (Serial myPort) { // starts reading data from the Serial Port
  // reads the data from the Serial Port up to the character '.' and puts it into the String variable "data".
  data = myPort.readStringUntil('.');
  data = data.substring(0,data.length()-1);
  index1 = data.indexOf(","); // find the character ',' and puts it into the variable "index1"
  angle= data.substring(0, index1); // read the data from position "0" to position of the variable index1 or thats the value of the angle the Arduino Board sent into the Serial Port
  distance= data.substring(index1+1, data.length()); // read the data from position "index1" to the end of the data pr thats the value of the distance
  // converts the String variables into Integer
  iAngle = int(angle);
  iDistance = int(distance);
}
void drawRadar()
  { pushMatrix();
  translate(width/2,height-height*0.074); // moves the starting coordinats to new location
  noFill();
  strokeWeight(2);
  stroke(98,245,31);
  // draws the arc lines
  arc(0,0,(width-width*0.0625),(width-width*0.0625),PI,TWO_PI);
  arc(0,0,(width-width*0.27),(width-width*0.27),PI,TWO_PI);
  arc(0,0,(width-width*0.479),(width-width*0.479),PI,TWO_PI);
  arc(0,0,(width-width*0.687),(width-width*0.687),PI,TWO_PI);
  // draws the angle lines
```

```
  line(-width/2,0,width/2,0);
  line(0,0,(-width/2)*cos(radians(30)),(-width/2)*sin(radians(30)));
  line(0,0,(-width/2)*cos(radians(60)),(-width/2)*sin(radians(60)));
  line(0,0,(-width/2)*cos(radians(90)),(-width/2)*sin(radians(90)));
  line(0,0,(-width/2)*cos(radians(120)),(-width/2)*sin(radians(120)));
  line(0,0,(-width/2)*cos(radians(150)),(-width/2)*sin(radians(150)));
  line((-width/2)*cos(radians(30)),0,width/2,0);
  popMatrix();
}
void drawObject()
  { pushMatrix();
  translate(width/2,height-height*0.074); // moves the starting coordinats to new location
  strokeWeight(9);
  stroke(255,10,10); // red color
  pixsDistance = iDistance*((height-height*0.1666)*0.025); // covers the distance from the sensor from
cm to pixels
  // limiting the range to 40 cms
  if(iDistance<40){
    // draws the object according to the angle and the distance
  line(pixsDistance*cos(radians(iAngle)),-pixsDistance*sin(radians(iAngle)),(width-
width*0.505)*cos(radians(iAngle)),-(width-width*0.505)*sin(radians(iAngle)));
  }
  popMatrix();
}
void drawLine()
  { pushMatrix();
  strokeWeight(9);
  stroke(30,250,60);
```

```
  translate(width/2,height-height*0.074); // moves the starting coordinats to new location
  line(0,0,(height-height*0.12)*cos(radians(iAngle)),-(height-height*0.12)*sin(radians(iAngle))); // draws the line according to the angle
  popMatrix();
}
void drawText() { // draws the texts on the screen
  pushMatrix();
  if(iDistance>40)
  {  noObject = "Out of
  Range";
  }
  else {
  noObject = "In Range";
  }
  fill(0,0,0);
  noStroke();
  rect(0, height-height*0.0648, width, height);
  fill(98,245,31);
  textSize(25);
  text("10cm",width-width*0.3854,height-height*0.0833);
  text("20cm",width-width*0.281,height-height*0.0833);
  text("30cm",width-width*0.177,height-height*0.0833);
  text("40cm",width-width*0.0729,height-height*0.0833);
  textSize(40);
  text("MES", width-width*0.8, height-height*0.03);
  text("Angle:" + iAngle +" °", width-width*0.45, height-height*0.03);
  text("Distance:", width-width*0.25, height-height*0.03);
  if(iDistance<40) {
```

```
  text("                    " + iDistance +" cm", width-width*0.225, height-height*0.0277);
  }
  textSize(25);
  fill(98,245,60);
  translate((width-width*0.4994)+width/2*cos(radians(30)),(height-height*0.0907)-
width/2*sin(radians(30)));
  rotate(-radians(-60));
  text("30°",0,0);
  resetMatrix();
  translate((width-width*0.503)+width/2*cos(radians(60)),(height-height*0.0888)-
width/2*sin(radians(60)));
  rotate(-radians(-30));
  text("60°",0,0);
  resetMatrix();
  translate((width-width*0.507)+width/2*cos(radians(90)),(height-height*0.0833)-
width/2*sin(radians(90)));
  rotate(radians(0));
  text("90°",0,0);
  resetMatrix();
  translate(width-width*0.513+width/2*cos(radians(120)),(height-height*0.07129)-
width/2*sin(radians(120)));
  rotate(radians(-30));
  text("120°",0,0);
  resetMatrix();
  translate((width-width*0.5104)+width/2*cos(radians(150)),(height-height*0.0574)-
width/2*sin(radians(150)));
  rotate(radians(-60));
  text("150°",0,0);
  popMatrix(); }
```